

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG



**BÁO CÁO
HỆ THỐNG GIÁM SÁT LƯU LƯỢNG
GIAO THÔNG**

Môn học: Chuyên đề Hệ thống thông tin

Thành viên: Nguyễn Đức Tâm – B21DCCN112

Lê Đức Thắng – B21DCCN664

Đỗ Đức Thành – B21DCCN676

Giảng viên: Thầy Nguyễn Trọng Khánh

Nhóm: 15

HÀ NỘI - 2025

LỜI CẢM ƠN

Trước tiên, chúng em xin bày tỏ lòng biết ơn sâu sắc đến thầy Nguyễn Trọng Khánh – giảng viên bộ môn "Chuyên đề Hệ thống thông tin". Em thật sự may mắn khi có cơ hội được học hỏi và nhận sự chỉ dẫn tận tình từ thầy trong suốt quá trình nghiên cứu tiểu luận này. Những kiến thức quý báu mà thầy truyền đạt không chỉ giúp em hoàn thiện bài tiểu luận mà còn trang bị cho em những kỹ năng cần thiết để nghiên cứu khoa học một cách bài bản và có hệ thống.

Cảm ơn thầy vì đã luôn sẵn sàng giải đáp thắc mắc, động viên em khi gặp khó khăn và khuyến khích em phát triển tư duy độc lập. Chính sự hỗ trợ và lòng nhiệt tình của thầy đã tạo động lực lớn cho em trong suốt quá trình thực hiện đề tài.

DANH MỤC CÁC THUẬT NGỮ, CHỮ VIẾT TẮT

Viết tắt	Tiếng Anh	Tiếng Việt
AI	Artificial Intelligence	Trí tuệ nhân tạo
YOLO	You Only Look Once	Thuật toán phát triển đối tượng theo thời gian thực
IoT	Internet of Things	Mạng lưới vạn vật kết nối
GPS	Global Positioning System	Hệ thống định vị toàn cầu
CCTV	Closed Circuit Television	Camera giám sát
HD	High Definition	Độ phân giải cao
V2X	Vehicle-to-Everything	Giao tiếp giữa các phương tiện và cơ sở hạ tầng
CNN	Convolutional Neural Network	Mạng nơ-ron tích chập cho nhận diện đối tượng
SSD	Single Shot Multibox Detector	Mạng nơ-ron tích chập phát hiện đối tượng
GPU	Graphics Processing Unit	Đơn vị xử lý đồ họa
CPU	Central Processing Unit	Đơn vị xử lý trung tâm
IoU	Intersection over Union	Giao điểm trên hợp nhất

DANH MỤC HÌNH ẢNH

Hình 1: Sơ đồ kiến trúc mạng YOLO.....	12
Hình 2: Các layer trong mạng darknet-53.	13
Hình 3: Kiến trúc một output của model YOLO.....	14
Hình 4: Các feature maps của mạng YOLOv3 với input shape là 416x416, output là 3 feature maps có kích thước lần lượt là 13x13, 26x26 và 52x52.	15
Hình 5: Xác định anchor box cho một vật thể.....	16
Hình 6: Khi 2 vật thể người và xe trùng mid point và cùng thuộc một cell. Thuật toán sẽ cần thêm những lượt tiebreak để quyết định đâu là class cho cell.	16
Hình 7: Công thức Bounding Box Regression Formula.....	17
Hình 8: Công thức ước lượng bounding box từ anchor box.....	18
Hình 9: Non-max suppression.....	18
Hình 10: Gán nhãn dữ liệu và chuyển đổi dữ liệu.....	20
Hình 11: Màn hình Google Colab	21

MỤC LỤC

LỜI CẢM ƠN.....	1
DANH MỤC CÁC THUẬT NGỮ, CHỮ VIẾT TẮT	2
DANH MỤC HÌNH ẢNH	3
MỤC LỤC	4
CHƯƠNG I: GIỚI THIỆU ĐỀ TÀI	6
GIỚI THIỆU CHUNG	6
MỤC TIÊU CỦA DỰ ÁN	6
QUY TRÌNH THỰC HIỆN	7
CHƯƠNG II: GIỚI THIỆU VỀ HỆ THỐNG	8
GIAO DIỆN HỆ THỐNG	8
a. Trang theo dõi lưu lượng 1 camera:	8
b. Trang chọn camera:	8
c. Trang xem một nhóm camera:	10
d. Trang xem lịch sử các hình ảnh đã được lưu theo thời gian thực:	10
CHƯƠNG III: THUẬT TOÁN YOLOV8.....	11
1. GIỚI THIỆU CHUNG	11
2. MẠNG YOLO LÀ GÌ?	11
3. CÁC PHIÊN BẢN	11
4. KIẾN TRÚC MẠNG YOLO [4].....	12
5. OUTPUT CỦA YOLO	13
6. DỰ BÁO TRÊN NHIỀU FEATURE MAP	14
7. ANCHOR BOX [5]	15
8. HÀM LOSS FUNCTION	17
9. DỰ BÁO BOUNDING BOX [6]	17
10. NON-MAX SUPPRESSION.....	18
CHƯƠNG IV: XÂY DỰNG MÔ HÌNH PHÁT HIỆN.....	20
CHUẨN BỊ DỮ LIỆU HUẤN LUYỆN.....	20
CÁC BƯỚC TẠO DỮ LIỆU	20
QUÁ TRÌNH HUẤN LUYỆN	20
PHÂN TÍCH KẾT QUẢ HUẤN LUYỆN	21
Các chỉ số đánh giá mô hình YOLO sau khi huấn luyện	21
• Precision (Độ chính xác).....	21
• Recall (Độ nhạy)	21
• Intersection over Union (IoU).....	22
• mAP@50 (Mean Average Precision tại IoU 0.5)	22
• mAP@50-95 (Mean Average Precision tại IoU từ 0.5 đến 0.95).....	22
• Kết quả Đánh giá Mô hình YOLO	23
Hiệu suất tổng thể.....	23
Hiệu suất theo lớp.....	23
Tổng kết	24
Thử nghiệm xác định đối tượng đã huấn luyện	24
CHƯƠNG V: XÂY DỰNG WEB GIÁM SÁT	25
1. GIỚI THIỆU CHUNG.....	25
2. THIẾT KẾ HỆ THỐNG WEB GIÁM SÁT	25
a. Kiến trúc hệ thống	25
b. Luồng dữ liệu.....	25
c. Cơ sở dữ liệu	26
3. GỬI LUỒNG DỮ LIỆU CAMERA TỚI BACKEND.....	26
a. Chức năng chính	26
b. Đoạn mã chính	26
c. Lợi ích và vai trò	26

4.	XÂY DỰNG BACKEND	26
a.	<i>Công nghệ sử dụng</i>	27
b.	<i>Chức năng chính</i>	27
c.	<i>Quy trình hoạt động</i>	27
d.	<i>Kết luận</i>	27
5.	GIAO DIỆN WEB	27
a.	<i>Trang dashboard</i>	27
b.	<i>Trang quản lý camera</i>	30
c.	<i>Trang xem camera theo nhóm</i>	31
d.	<i>Trang xem các hình ảnh camera chụp lại</i>	33
CHƯƠNG VI: KẾT QUẢ		35
1.	KẾT QUẢ ĐẠT ĐƯỢC	35
a.	<i>Về mặt lý thuyết</i>	35
b.	<i>Về mặt thực tiễn</i>	35
2.	HẠN CHẾ	35
3.	HƯỚNG PHÁT TRIỂN	35
DANH MỤC TÀI LIỆU THAM KHẢO		37

CHƯƠNG I: GIỚI THIỆU ĐỀ TÀI

Giới thiệu chung

Trong thời đại công nghệ số, việc ứng dụng trí tuệ nhân tạo (AI) vào các lĩnh vực đời sống đang ngày càng phổ biến, đặc biệt là trong giao thông – một lĩnh vực đòi hỏi sự chính xác, nhanh chóng và hiệu quả cao. Các hệ thống giám sát giao thông truyền thống thường phụ thuộc nhiều vào con người và không thể theo dõi liên tục trong thời gian thực, dẫn đến nhiều hạn chế trong việc phát hiện và xử lý các tình huống như ùn tắc, tai nạn, hoặc vi phạm luật giao thông.

Để khắc phục các vấn đề trên, các giải pháp giám sát thông minh sử dụng thị giác máy tính và mô hình học sâu đã được phát triển. Trong đó, mô hình YOLO (You Only Look Once) là một trong những thuật toán nổi bật trong việc phát hiện đối tượng trong ảnh và video thời gian thực.

Đề tài “**Hệ thống giám sát lưu lượng giao thông**” được thực hiện nhằm nghiên cứu và ứng dụng mô hình YOLOv8 để phát hiện, phân loại các phương tiện giao thông như xe máy, ô tô, xe buýt, xe tải từ hình ảnh hoặc video camera, đồng thời lưu trữ và hiển thị kết quả nhận diện trên giao diện web.

Mục tiêu của dự án

Dự án xây dựng **hệ thống giám sát lưu lượng giao thông** nhằm ứng dụng công nghệ thị giác máy tính để tự động nhận diện và thống kê số lượng phương tiện giao thông theo thời gian thực, giúp cải thiện hiệu quả quản lý và quy hoạch hạ tầng giao thông đô thị. Hệ thống hướng đến việc thay thế quy trình thống kê thủ công truyền thống, vốn tốn nhiều công sức và thiếu chính xác, bằng một giải pháp tự động, nhanh chóng và có độ tin cậy cao.

Một trong những mục tiêu cốt lõi của dự án là phát triển quy trình xử lý dữ liệu thời gian thực thông qua kiến trúc mô-đun bao gồm ba lớp chính: **nhận diện hình ảnh - lưu trữ dữ liệu - trực quan hóa kết quả**. Hệ thống sử dụng mô hình YOLOv8 để nhận diện và phân loại các loại phương tiện như xe máy, ô tô, xe tải, xe buýt từ video camera giám sát. Kết quả được lưu trữ có hệ thống vào cơ sở dữ liệu quan hệ (MySQL), kèm theo các thông tin về thời gian và địa điểm ghi nhận, nhằm phục vụ cho mục đích phân tích sau này.

Dữ liệu thu thập sẽ được phân tích và hiển thị trên giao diện dashboard trực quan, giúp người quản lý giao thông dễ dàng theo dõi mật độ xe cộ tại từng thời điểm, phát hiện các khu vực có nguy cơ ùn tắc cao hoặc thời điểm cao điểm trong ngày. Hệ thống có thể mở rộng để cung cấp báo cáo định kỳ theo ngày, tuần hoặc tháng, hỗ trợ đưa ra các quyết định điều phối giao thông hoặc cải thiện hạ tầng dựa trên số liệu thực tế.

Bên cạnh đó, dự án còn hướng đến việc nâng cao trải nghiệm người dùng thông qua một giao diện web hiện đại, có thể hoạt động mượt mà trên cả máy tính và thiết bị di động. Các công nghệ được sử dụng bao gồm Python (Flask), mô hình học sâu (YOLOv8), cơ sở dữ liệu MySQL, để đảm bảo khả năng mở rộng, dễ bảo trì và thân thiện với người sử dụng.

Thông qua hệ thống này, các cơ quan chức năng và nhà quản lý có thể tiếp cận thông tin giao thông một cách chủ động, chính xác và kịp thời, góp phần nâng cao chất lượng điều hành và giảm thiểu ùn tắc trong môi trường đô thị hiện đại.

Quy trình thực hiện

Quá trình phát triển hệ thống giám sát lưu lượng giao thông được triển khai theo một lộ trình khoa học, bao gồm các giai đoạn từ nghiên cứu lý thuyết đến thiết kế, hiện thực hóa, kiểm thử và đề xuất mở rộng hệ thống. Cụ thể như sau:

Giai đoạn 1: Tìm hiểu kiến thức nền tảng: Nhóm bắt đầu bằng việc nghiên cứu các lĩnh vực liên quan đến thị giác máy tính (Computer Vision), học sâu (Deep Learning) và đặc biệt là mô hình nhận diện đối tượng YOLO (You Only Look Once), phiên bản YOLOv8. Đồng thời, nhóm cũng tìm hiểu các công nghệ phát triển web như Node.js, Express.js, MySQL nhằm chuẩn bị kiến thức cho việc xây dựng hệ thống backend xử lý dữ liệu một cách hiệu quả.

Giai đoạn 2: Phân tích và thiết kế hệ thống: Sau khi nắm vững công nghệ, nhóm tiến hành phân tích yêu cầu và thiết kế kiến trúc tổng thể của hệ thống, bao gồm các thành phần chính: mô hình trí tuệ nhân tạo (AI) nhận diện và phân loại phương tiện giao thông, backend xử lý dữ liệu và giao tiếp với cơ sở dữ liệu, cơ sở dữ liệu MySQL lưu trữ thông tin về lưu lượng phương tiện, giao diện hiển thị kết quả giám sát. Kiến trúc hệ thống được thiết kế để đảm bảo tính chính xác, mở rộng và dễ bảo trì.

Giai đoạn 3: Xây dựng hệ thống:

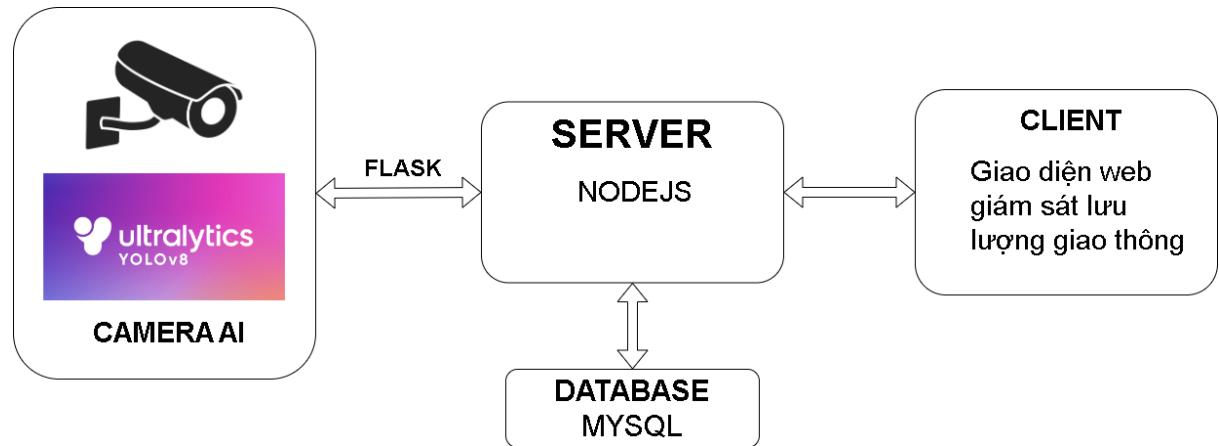
- Huấn luyện và triển khai mô hình YOLOv8 để nhận diện các phương tiện trong video thời gian thực.
- Xây dựng server backend bằng Node.js kết nối với cơ sở dữ liệu MySQL để lưu trữ dữ liệu nhận diện.
- Xây dựng giao diện web đơn giản nhằm trực quan hóa số liệu và hình ảnh video đã xử lý.

Giai đoạn 4: Kiểm thử và đánh giá: Hệ thống được kiểm thử với nhiều nguồn video khác nhau để đánh giá các yếu tố: độ chính xác của mô hình AI trong điều kiện ánh sáng, góc quay khác nhau; hiệu năng của hệ thống backend khi xử lý và lưu trữ dữ liệu liên tục; tính trực quan và dễ sử dụng của giao diện người dùng. Kết quả kiểm thử giúp nhóm phát hiện lỗi, điều chỉnh tham số và tối ưu hiệu suất tổng thể của hệ thống.

Giai đoạn 5: Tổng kết và định hướng phát triển: Sau khi hệ thống hoạt động ổn định, nhóm tiến hành tổng kết kết quả đạt được, đánh giá hiệu quả triển khai. Đồng thời, nhóm cũng đề xuất một số hướng phát triển trong tương lai như: tích hợp tính năng nhận diện biển số xe; phát hiện hành vi vi phạm giao thông (đi sai làn, vượt đèn đỏ,...); tích hợp nhiều camera giám sát cùng lúc; triển khai hệ thống trên nền tảng IoT hoặc điện toán đám mây để nâng cao khả năng ứng dụng thực tế.

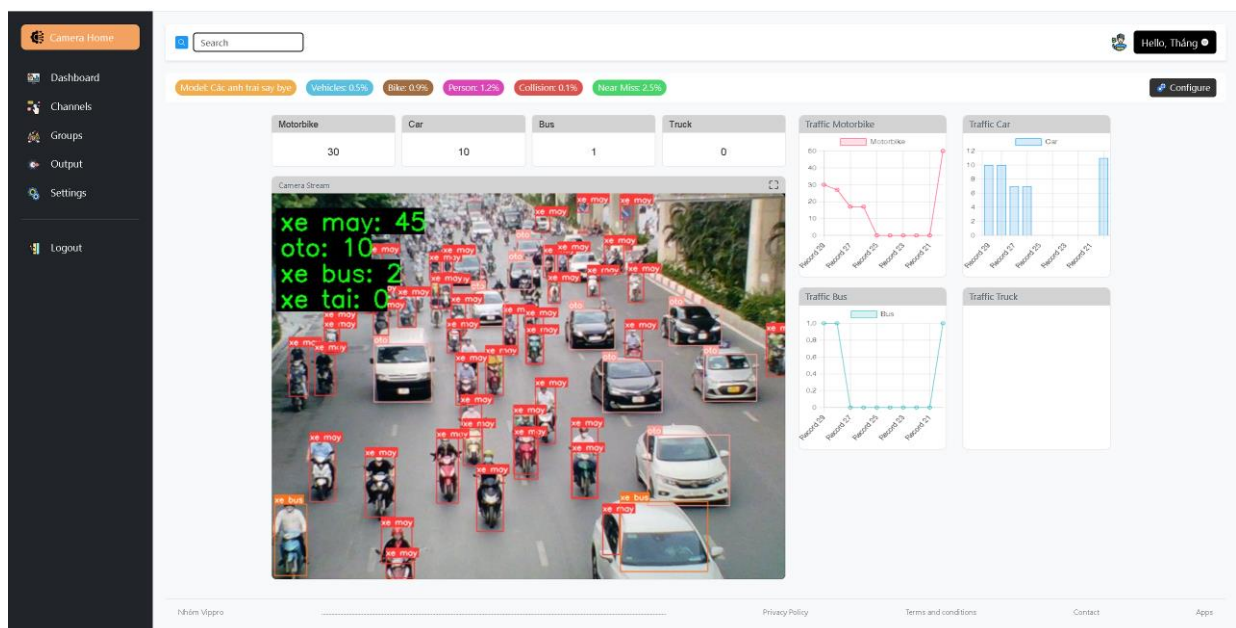
CHƯƠNG II: GIỚI THIỆU VỀ HỆ THỐNG

TỔNG QUAN CẤU TRÚC VỀ HỆ THỐNG



GIAO DIỆN HỆ THỐNG

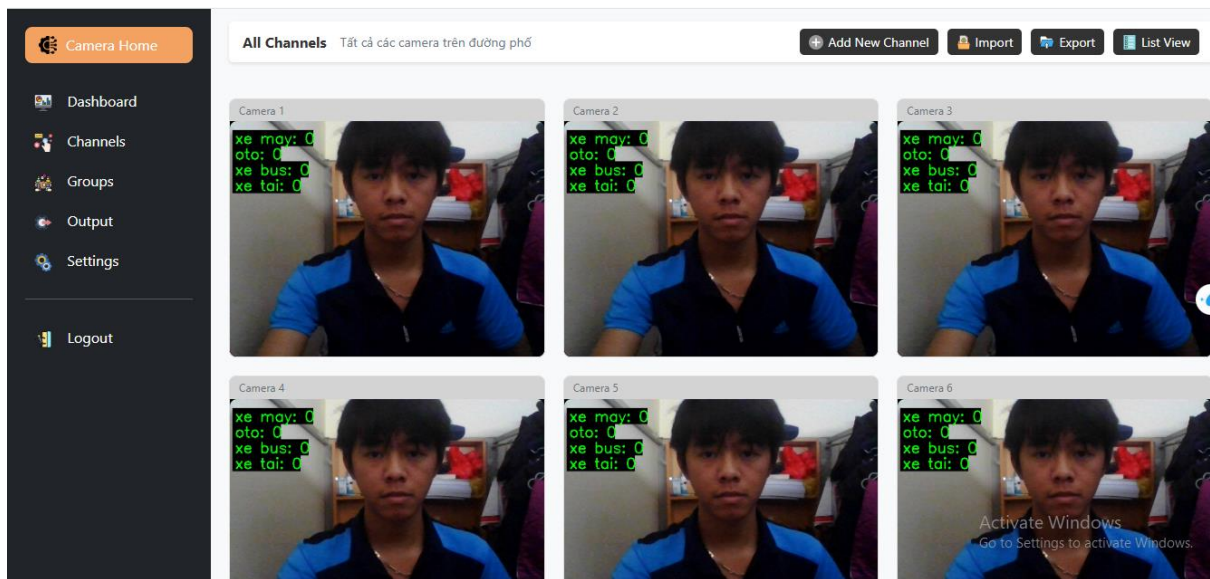
a. Trang theo dõi lưu lượng 1 camera:



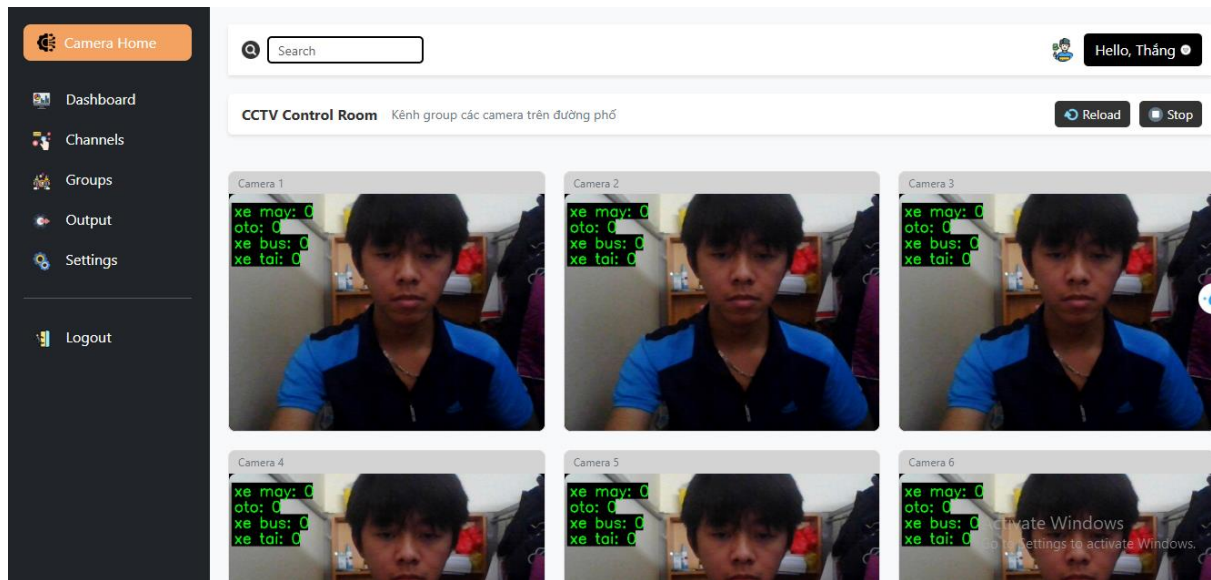
Hiện thị camera hiện tại được chọn.

Biểu đồ số phương tiện các loại theo thời gian thực.

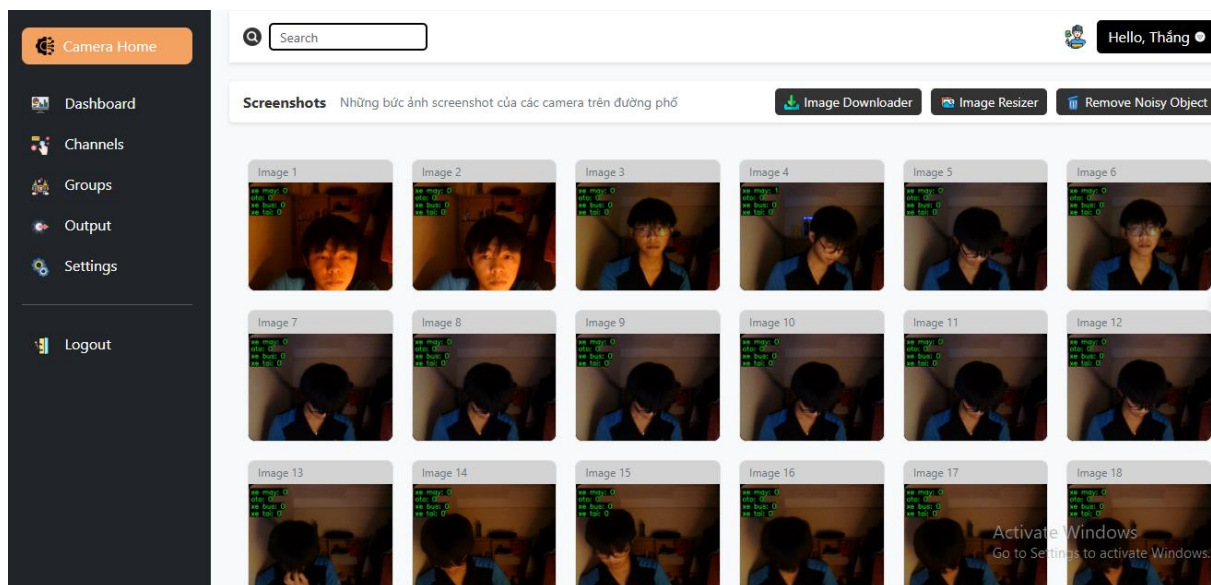
b. Trang chọn camera:



c. Trang xem một nhóm camera:



d. Trang xem lịch sử các hình ảnh đã được lưu theo thời gian thực:



CHƯƠNG III: THUẬT TOÁN YOLOv8

1. Giới thiệu chung

Có lẽ trong vài năm trở lại đây, object detection là một trong những đề tài rất hot của deep learning bởi khả năng ứng dụng cao, dữ liệu dễ chuẩn bị và kết quả ứng dụng thì cực kì nhiều. Các thuật toán mới của object detection như YOLO, SSD có tốc độ khá nhanh và độ chính xác cao nên giúp cho Object Detection có thể thực hiện được các tác vụ dường như là real time, thậm chí là nhanh hơn so với con người mà độ chính xác không giảm. Các mô hình cũng trở nên nhẹ hơn nên có thể hoạt động trên các thiết bị IoT để tạo nên các thiết bị thông minh. [1]

Ngoài ra, một thuật toán object detection có thể tạo ra những ứng dụng rất đa dạng như: Đếm số lượng vật thể, thanh toán tiền tại quầy hàng, chấm công tự động, phát hiện vật thể nguy hiểm như súng, dao,... và rất nhiều các ứng dụng khác. Có thể nói dường như bất kì lĩnh vực nào cũng đều có thể ứng dụng được object detection.

Bên cạnh đó nguồn dữ liệu ảnh lại vô cùng đa dạng và sẵn có vì chỉ cần google là tìm được tất cả những gì ta cần. Đó cũng là một ưu điểm để huấn luyện model object detection.

Chính vì tính ứng dụng rất cao, dễ chuẩn bị dữ liệu và huấn luyện mô hình đơn giản nên chúng em xin giới thiệu thuật toán object detection state-of-art, đó chính là YOLO.

2. Mạng YOLO là gì?

YOLO là thuật toán object detection nên mục tiêu của mô hình không chỉ là dự báo nhãn cho vật thể như các bài toán classification mà nó còn xác định location của vật thể. Do đó YOLO có thể phát hiện được nhiều vật thể có nhãn khác nhau trong một bức ảnh thay vì chỉ phân loại duy nhất một nhãn cho một bức ảnh.

Sở dĩ YOLO có thể phát hiện được nhiều vật thể trên một bức ảnh như vậy là vì thuật toán có những cơ chế rất đặc biệt.

3. Các phiên bản

YOLOv1: Phiên bản đầu tiên, giới thiệu ý tưởng cơ bản.

YOLOv3: Tối ưu hóa cho việc phát hiện đối tượng nhỏ và cải thiện độ chính xác tổng thể. [3]

YOLOv4: Sử dụng các kỹ thuật tối ưu hóa như Mosaic data augmentation, Self-adversarial training, và CSPNet để cải thiện khả năng phát hiện. Đạt hiệu suất tốt trên nhiều bài kiểm tra với các kích thước khác nhau của đối tượng. Có khả năng chạy nhanh trên GPU với hiệu suất cao.

YOLOv5: Được viết bằng PyTorch, cho phép dễ dàng tùy chỉnh và triển khai. Nhiều kích thước mô hình (small, medium, large) giúp người dùng chọn lựa tùy theo nhu cầu tài nguyên và độ chính xác. Cải thiện tốc độ và độ chính xác so với YOLOv4 trong nhiều tình huống thực tế.

YOLOv6: Thiết kế cho ứng dụng trong thực tế, với cải tiến về độ chính xác và tốc độ. Sử dụng EfficientNet và các cấu trúc mạng mới để cải thiện khả năng phát hiện đối tượng nhỏ.

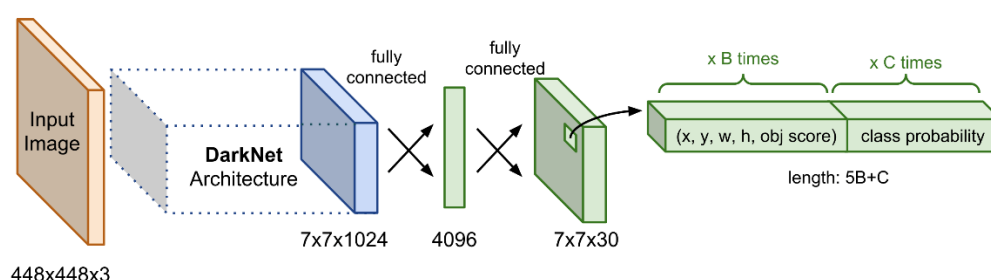
YOLOv7: Sử dụng các kỹ thuật như Dynamic Label Assignment và Model Scaling để tối ưu hóa mô hình cho cả tốc độ và độ chính xác. Đạt được nhiều kết quả tốt trong các bài kiểm tra trên COCO dataset và các bài toán thực tế.

YOLOv8: Tối ưu hóa cho khả năng phát hiện và phân loại đối tượng trong nhiều tình huống khác nhau. Có khả năng chạy trên cả CPU và GPU, với hiệu suất tốt trên cả hai. Cải thiện hơn nữa về độ chính xác, đặc biệt là với các đối tượng nhỏ và chồng chéo.

4. Kiến trúc mạng YOLO [4]

Kiến trúc YOLO bao gồm: base network là các mạng convolution làm nhiệm vụ trích xuất đặc trưng. Phần phía sau là những Extra Layers được áp dụng để phát hiện vật thể trên feature map của base network.

Base network của YOLO sử dụng chủ yếu là các convolutional layer và các fully connected layer. Các kiến trúc YOLO cũng khá đa dạng và có thể tùy biến thành các version cho nhiều input shape khác nhau.



Hình 1: Sơ đồ kiến trúc mạng YOLO.

Thành phần Darknet Architecture được gọi là base network có tác dụng trích xuất đặc trưng. Output của base network là một feature map có kích thước 7x7x1024 sẽ được sử dụng làm input cho các Extra layers có tác dụng dự đoán nhãn và tọa độ bounding box của vật thể.

Trong YOLO version 3 tác giả áp dụng một mạng feature extractor là darknet-53. Mạng này gồm 53 convolutional layers kết nối liên tiếp, mỗi layer được theo sau bởi một batch normalization và một activation Leaky Relu. Để giảm kích thước của output sau mỗi convolution layer, tác giả down sample bằng các filter với kích thước là 2. Mạng này có tác dụng giảm thiểu số lượng tham số cho mô hình.

	Type	Filters	Size	Output
	Convolutional	32	3×3	256×256
	Convolutional	64	$3 \times 3 / 2$	128×128
1x	Convolutional	32	1×1	
	Convolutional	64	3×3	
	Residual			128×128
	Convolutional	128	$3 \times 3 / 2$	64×64
2x	Convolutional	64	1×1	
	Convolutional	128	3×3	
	Residual			64×64
	Convolutional	256	$3 \times 3 / 2$	32×32
8x	Convolutional	128	1×1	
	Convolutional	256	3×3	
	Residual			32×32
	Convolutional	512	$3 \times 3 / 2$	16×16
8x	Convolutional	256	1×1	
	Convolutional	512	3×3	
	Residual			16×16
	Convolutional	1024	$3 \times 3 / 2$	8×8
4x	Convolutional	512	1×1	
	Convolutional	1024	3×3	
	Residual			8×8
	Avgpool		Global	
	Connected		1000	
	Softmax			

Hình 2: Các layer trong mạng darknet-53.

Các bức ảnh khi được đưa vào mô hình sẽ được scale để về chung một kích thước phù hợp với input shape của mô hình và sau đó được gom lại thành batch đưa vào huấn luyện.

Hiện tại YOLO đang hỗ trợ 2 đầu vào chính là 416×416 và 608×608 . Mỗi một đầu vào sẽ có một thiết kế các layers riêng phù hợp với shape của input. Sau khi đi qua các layer convolutional thì shape giảm dần theo cấp số nhân là 2. Cuối cùng ta thu được một feature map có kích thước tương đối nhỏ để dự báo vật thể trên từng ô của feature map.

Kích thước của feature map sẽ phụ thuộc vào đầu vào. Đối với input 416×416 thì feature map có các kích thước là 13×13 , 26×26 và 52×52 . Và khi input là 608×608 sẽ tạo ra feature map 19×19 , 38×38 , 72×72 .

5. Output của YOLO

Output của mô hình YOLO là một véc tơ sẽ bao gồm các thành phần:

$$yT = [p0, \langle tx, ty, tw, th \rangle_{\text{bounding box}}, \langle p1, p2, \dots, pc \rangle_{\text{scores of c classes}}]$$

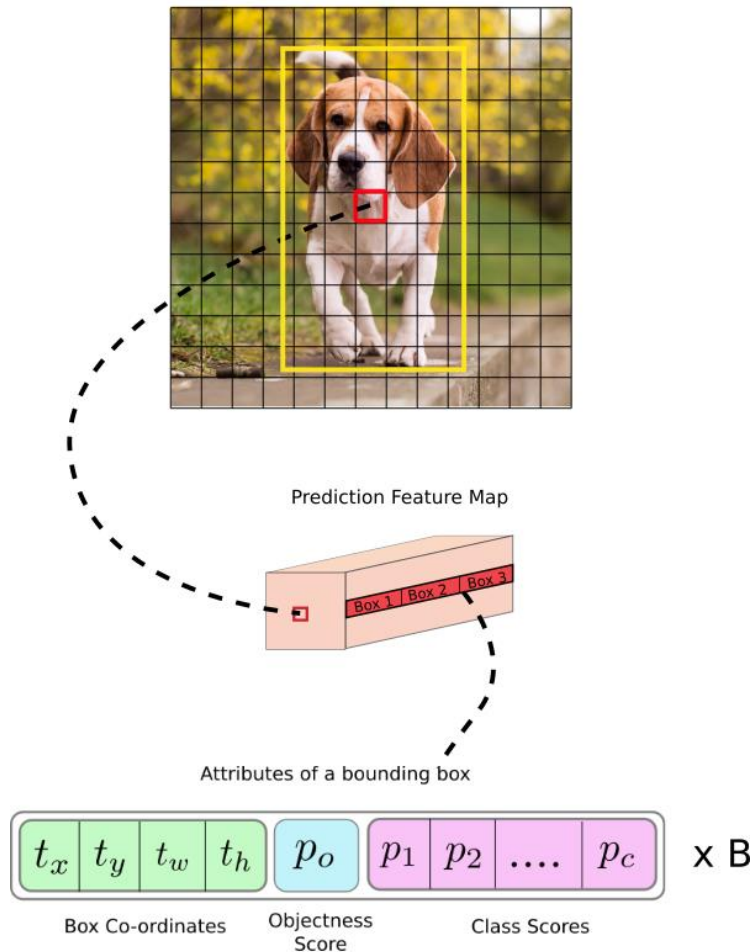
Trong đó:

- $p0$ là xác suất dự báo vật thể xuất hiện trong bounding box.
- $\langle tx, ty, tw, th \rangle_{\text{bounding box}}$ giúp xác định bounding box. Trong đó tx, ty là tọa độ tâm và tw, th là kích thước rộng, dài của bounding box.
- $\langle p1, p2, \dots, pc \rangle_{\text{scores of c classes}}$ là véc tơ phân phối xác suất dự báo của các classes.

Việc hiểu output khá là quan trọng để ta cấu hình tham số chuẩn xác khi huấn luyện model qua các open source như darknet. Như vậy output sẽ được xác định theo

số lượng classes theo công thức $(n_class+5)$. Nếu huấn luyện 80 classes thì sẽ có output là 85. Trường hợp áp dụng 3 anchors/cell thì số lượng tham số output sẽ là:

$$(n_class+5) \times 3 = 85 \times 3 = 255$$

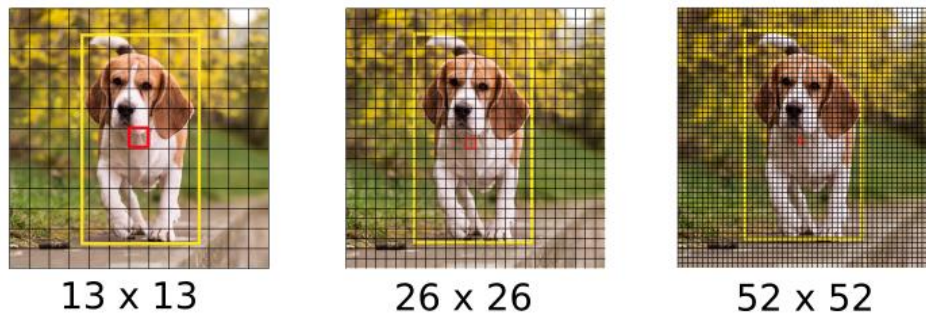


Hình 3: Kiến trúc một output của model YOLO

Hình ảnh gốc là một feature map kích thước 13x13. Trên mỗi một cell của feature map ta lựa chọn ra 3 anchor boxes với kích thước khác nhau lần lượt là Box 1, Box 2, Box 3 sao cho tâm của các anchor boxes trùng với cell. Khi đó output của YOLO là một véc tơ concatenate của 3 bounding boxes. Các attributes của một bounding box được mô tả như dòng cuối cùng trong hình.

6. Dự báo trên nhiều feature map

Cũng tương tự như SSD, YOLOv3 dự báo trên nhiều feature map. Những feature map ban đầu có kích thước nhỏ giúp dự báo được các object kích thước lớn. Những feature map sau có kích thước lớn hơn trong khi anchor box được giữ cố định kích thước nên sẽ giúp dự báo các vật thể kích thước nhỏ.



Hình 4: Các feature maps của mạng YOLOv3 với input shape là 416×416 , output là 3 feature maps có kích thước lần lượt là 13×13 , 26×26 và 52×52 .

Trên mỗi một cell của các feature map ta sẽ áp dụng 3 anchor box để dự đoán vật thể. Như vậy số lượng các anchor box khác nhau trong một mô hình YOLO sẽ là 9 (3 feature map x 3 anchor box).

Đồng thời trên một feature map hình vuông $S \times S$, mô hình YOLOv3 sinh ra một số lượng anchor box là: $S \times S \times 3$. Như vậy số lượng anchor boxes trên một bức ảnh sẽ là:

$$(13 \times 13 + 26 \times 26 + 52 \times 52) \times 3 = 10647 \text{ (anchor boxes)}$$

Đây là một số lượng rất lớn và là nguyên nhân khiến quá trình huấn luyện mô hình YOLO vô cùng chậm bởi ta cần dự báo đồng thời nhãn và bounding box trên đồng thời 10647 bounding boxes.

Một số lưu ý khi huấn luyện YOLO:

Khi huấn luyện YOLO sẽ cần phải có RAM dung lượng lớn hơn để save được 10647 bounding boxes như trong kiến trúc này.

Không thể thiết lập các batch_size quá lớn như trong các mô hình classification vì rất dễ Out of memory. Package darknet của YOLO đã chia nhỏ một batch thành các subdivisions cho vừa với RAM.

Thời gian xử lý của một step trên YOLO lâu hơn rất rất nhiều lần so với các mô hình classification. Do đó nên thiết lập steps giới hạn huấn luyện cho YOLO nhỏ. Đối với các tác vụ nhận diện dưới 5 classes, dưới 5000 steps là có thể thu được nghiệm tạm chấp nhận được. Các mô hình có nhiều classes hơn có thể tăng số lượng steps theo cấp số nhân.

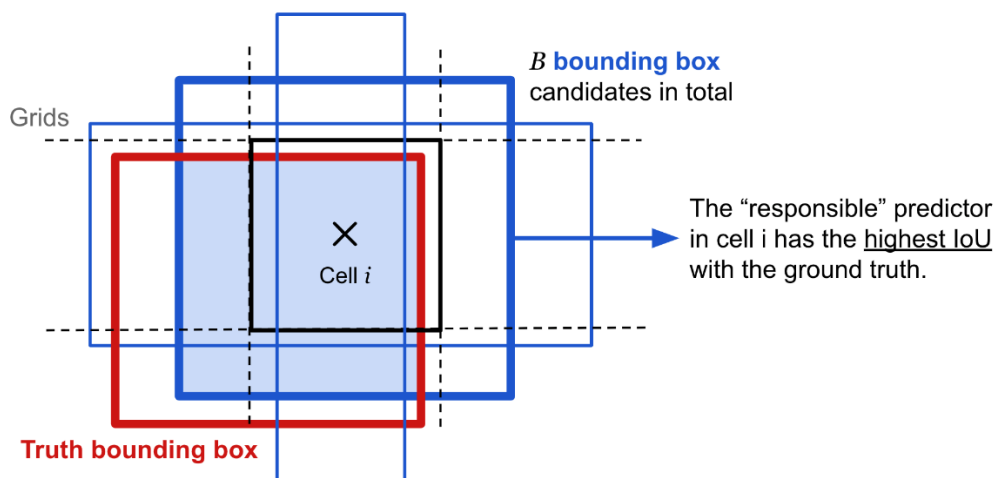
7. Anchor box [5]

Để tìm được bounding box cho vật thể, YOLO sẽ cần các anchor box làm cơ sở ước lượng. Những anchor box này sẽ được xác định trước và sẽ bao quanh vật thể một cách tương đối chính xác. Sau này thuật toán regression bounding box sẽ tinh chỉnh lại anchor box để tạo ra bounding box dự đoán cho vật thể. Trong một mô hình YOLO:

- Mỗi một vật thể trong hình ảnh huấn luyện được phân bổ về một anchor box. Trong trường hợp có từ 2 anchor boxes trở lên cùng bao quanh vật thể thì ta sẽ xác định anchor box mà có IoU với ground truth bounding box là cao nhất.

- Mỗi một vật thể trong hình ảnh huấn luyện được phân bổ về một cell trên feature map mà chứa điểm mid point của vật thể. Chẳng hạn như hình chú chó trong hình 3 sẽ

được phân về cho cell màu đỏ vì điểm mid point của ảnh chú chó rơi vào đúng cell này. Từ cell ta sẽ xác định các anchor boxes bao quanh hình ảnh chú chó.



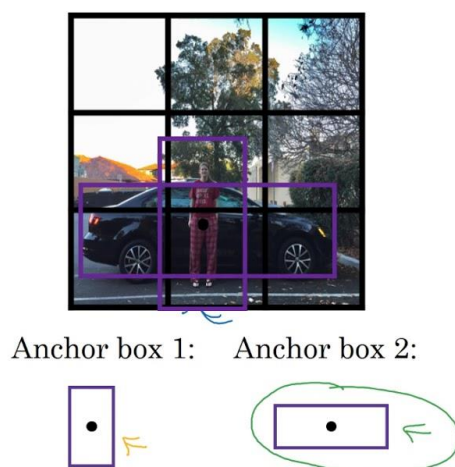
Hình 5: Xác định anchor box cho một vật thể

- Từ cell i ta xác định được 3 anchor boxes viền xanh như trong hình. Cả 3 anchor boxes này đều giao nhau với bounding box của vật thể. Tuy nhiên chỉ anchor box có đường viền dày nhất màu xanh được lựa chọn làm anchor box cho vật thể bởi nó có IoU so với ground truth bounding box là cao nhất.

Như vậy khi xác định một vật thể ta sẽ cần xác định 2 thành phần gắn liền với nó là (cell, anchor box). Không chỉ riêng mình cell hoặc chỉ mình anchor box.

Một số trường hợp 2 vật thể bị trùng mid point, mặc dù rất hiếm khi xảy ra, thuật toán sẽ rất khó xác định được class cho chúng.

Anchor box example



$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \\ p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

Handwritten notes next to the vector y show two columns of values: one with '1' and '0' and another with '0' and '1', representing different classes or states.

Andrew Ng

Hình 6: Khi 2 vật thể người và xe trùng mid point và cùng thuộc một cell. Thuật toán sẽ cần thêm những lượt tiebreak để quyết định đâu là class cho cell.

8. Hàm loss function

Cũng tương tự như SSD, hàm loss function của YOLO chia thành 2 phần: Lloc (localization loss) đo lường sai số của bounding box và Lcls (confidence loss) đo lường sai số của phân phối xác suất các classes.

- Lloc là hàm mất mát của bounding box dự báo so với thực tế.

- Lcls là hàm mất mát của phân phối xác suất. Trong đó tổng đầu tiên là mất mát của dự đoán có vật thể trong cell hay không? Và tổng thứ 2 là mất mát của phân phối xác suất nếu có vật thể trong cell.

Ngoài ra để điều chỉnh phạt loss function trong trường hợp dự đoán sai bounding box ta thông qua hệ số điều chỉnh λ_{coord} và ta muốn giảm nhẹ hàm loss function trong trường hợp cell không chứa vật thể bằng hệ số điều chỉnh λ .

9. Dự báo bounding box [6]

Để dự báo bounding box cho một vật thể ta dựa trên một phép biến đổi từ anchor box và cell.

YOLOv2 và YOLOv3 dự đoán bounding box sao cho nó sẽ không lệch khỏi vị trí trung tâm quá nhiều. Nếu bounding box dự đoán có thể đặt vào bất kỳ phần nào của hình ảnh, như trong mạng regional proposal network, việc huấn luyện mô hình có thể trở nên không ổn định.

Cho một anchor box có kích thước (pw,ph) tại cell nằm trên feature map với góc trên cùng bên trái của nó là (cx,cy), mô hình dự đoán 4 tham số (tx,ty,tw,th) trong đó 2 tham số đầu là độ lệch (offset) so với góc trên cùng bên trái của cell và 2 tham số sau là tỷ lệ so với anchor box. Và các tham số này sẽ giúp xác định bounding box dự đoán b có tâm (bx,by) và kích thước (bw,bh) thông qua hàm sigmoid và hàm exponential như các công thức bên dưới:

$$b_x = \sigma(t_x) + c_x$$

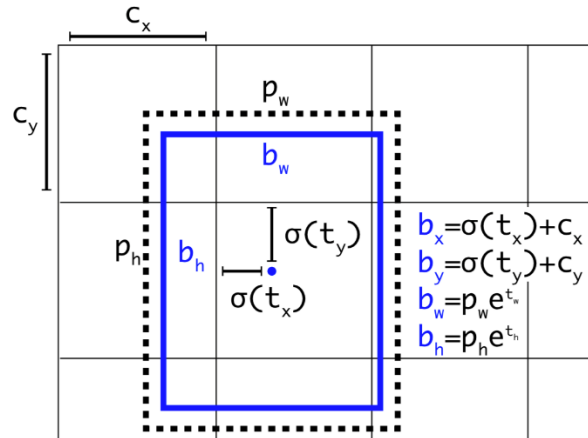
$$b_y = \sigma(t_y) + c_y$$

$$b_w = p_w e^{t_w}$$

$$b_h = p_h e^{t_h}$$

Hình 7: Công thức Bounding Box Regression Formula

Ngoài ra do các tọa độ đã được hiệu chỉnh theo width và height của bức ảnh nên luôn có giá trị nằm trong ngưỡng [0, 1]. Do đó khi áp dụng hàm sigmoid giúp ta giới hạn được tọa độ không vượt quá xa các ngưỡng này.



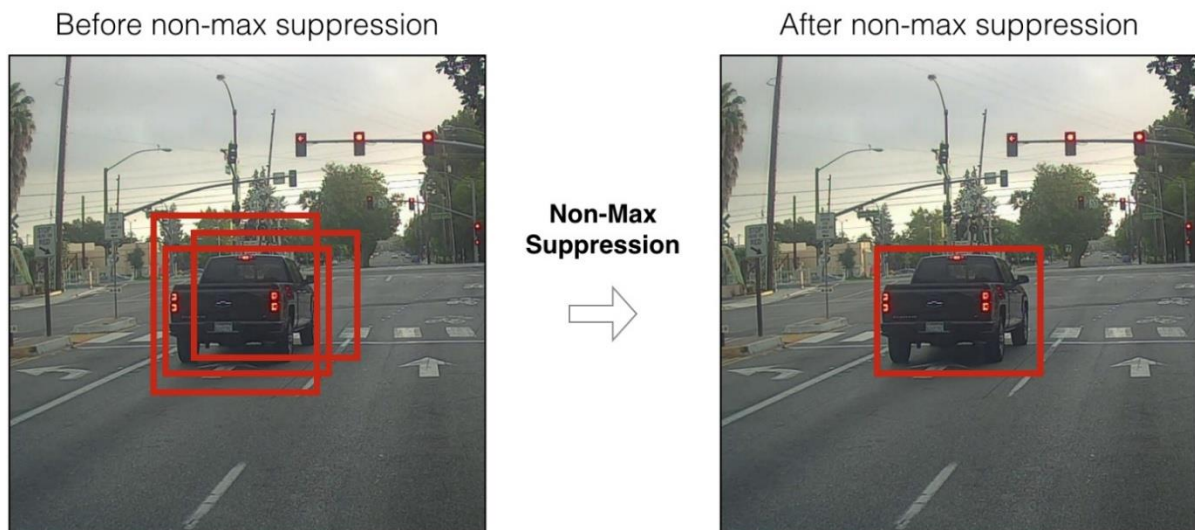
Hình 8: Công thức ước lượng bounding box từ anchor box

Giải thích hình ảnh:

- Hình chữ nhật nét đứt bên ngoài là anchor box có kích thước là (p_w, p_h) .
- Tọa độ của một bounding box sẽ được xác định dựa trên đồng thời cả anchor box và cell mà nó thuộc về. Điều này giúp kiểm soát vị trí của bounding box dự đoán đầu đó quanh vị trí của cell và bounding box mà không vượt quá xa ra bên ngoài giới hạn này.
- Do đó quá trình huấn luyện sẽ ổn định hơn rất nhiều so với YOLO version 1.s.

10. Non-max suppression

Do thuật toán YOLO dự báo ra rất nhiều bounding box trên một bức ảnh nên đối với những cell có vị trí gần nhau, khả năng các khung hình bị overlap là rất cao. Trong trường hợp đó YOLO sẽ cần đến non-max suppression để giảm bớt số lượng các khung hình được sinh ra một cách đáng kể.



Hình 9: Non-max suppression

Các bước của non-max suppression:

- Step 1: Đầu tiên ta sẽ tìm cách giảm bớt số lượng các bounding box bằng cách lọc bỏ toàn bộ những bounding box có xác suất chứa vật thể nhỏ hơn một ngưỡng threshold nào đó, thường là 0.5.

- Step 2: Đối với các bounding box giao nhau, non-max suppression sẽ lựa chọn ra một bounding box có xác suất chứa vật thể là lớn nhất. Sau đó tính toán chỉ số giao thoa IoU với các bounding box còn lại.

Nếu chỉ số này lớn hơn ngưỡng threshold thì điều đó chứng tỏ 2 bounding boxes đang overlap nhau rất cao. Ta sẽ xóa các bounding có xác suất thấp hơn và giữ lại bounding box có xác suất cao nhất. Cuối cùng, ta thu được một bounding box duy nhất cho một vật thể.

CHƯƠNG IV: XÂY DỰNG MÔ HÌNH PHÁT HIỆN

Chuẩn bị dữ liệu huấn luyện

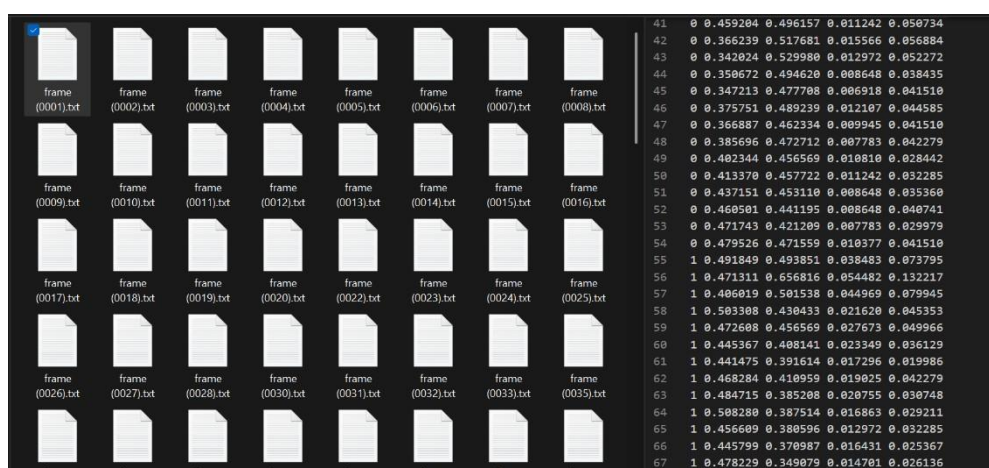
Để huấn luyện mô hình YOLOv8, ta cần một tập dữ liệu hình ảnh đã gán nhãn, trong đó mỗi hình ảnh có các chú thích xác định nhân và tọa độ hộp giới hạn của các đối tượng. Dữ liệu được gán nhãn này là yếu tố quan trọng để huấn luyện mô hình phát hiện chính xác đối tượng trong các hình ảnh mới. [7]

Các bước tạo dữ liệu

Thu thập Dữ liệu: Xác định vấn đề và loại dữ liệu cần thiết. Khi đã xác định, tiến hành thu thập dữ liệu. Điều này có thể bao gồm việc chụp ảnh, tải hình ảnh từ Internet hoặc thu thập từ các nguồn khác.

Gán nhãn Dữ liệu: Sau khi thu thập dữ liệu, tiến hành gán nhãn bằng cách chú thích vị trí và loại của các đối tượng trong hình ảnh. Có nhiều công cụ hỗ trợ việc gán nhãn dữ liệu.

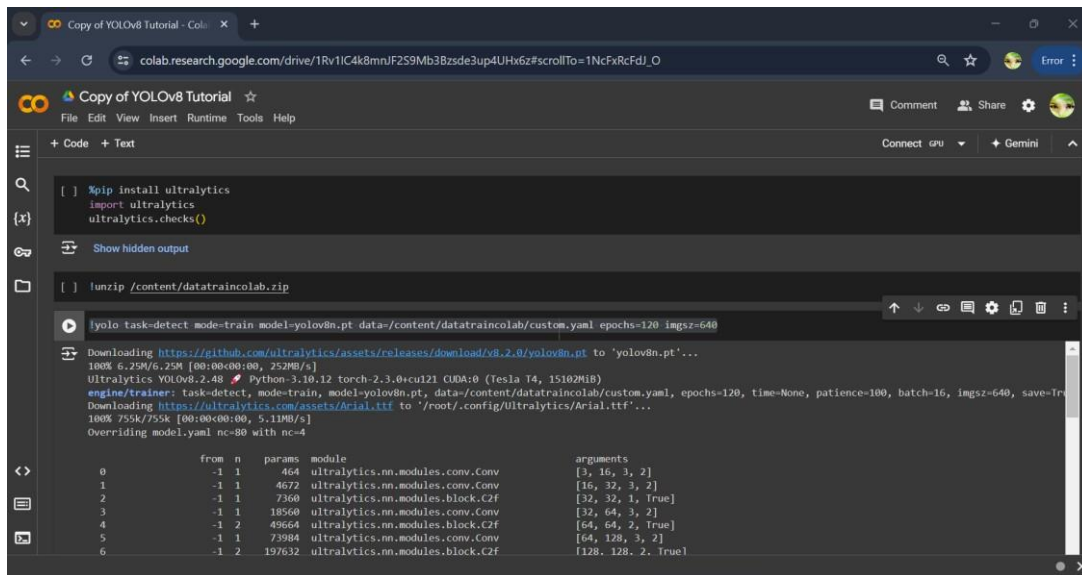
Chuyển đổi Dữ liệu: Chuyển đổi kích thước tất cả hình ảnh về độ phân giải 640px. Bước này giúp tạo sự đồng nhất và chuẩn bị dữ liệu cho việc huấn luyện.



Hình 10: Gán nhãn dữ liệu và chuyển đổi dữ liệu

Quá trình huấn luyện

Sử dụng Google Colab: Sử dụng Google Colab để tận dụng tài nguyên GPU và CPU miễn phí trong quá trình huấn luyện.



Hình 11: Màn hình Google Colab

Thiết lập Tham số Huấn luyện: Thiết lập tham số huấn luyện là bước quan trọng vì nó ảnh hưởng đến hiệu suất, độ chính xác và thời gian huấn luyện của mô hình:

- **Batch Size:** Số mẫu được xử lý qua mạng neural trong một lần. Batch size phù hợp phụ thuộc vào kích thước dữ liệu, độ phức tạp mô hình, và tài nguyên tính toán sẵn có. Giá trị mặc định thường là 64.

- **Số lượng Epochs:** Tham số này quy định số lần toàn bộ dữ liệu huấn luyện được truyền qua mô hình. Tăng số lượng epochs có thể cải thiện hiệu suất của mô hình, nhưng cần cân bằng để tránh quá khớp. Thường dao động từ 100 đến 150 epochs. Ví dụ, khi đặt epochs=120, mô hình sẽ được huấn luyện qua tập dữ liệu 120 lần.

Phân tích kết quả huấn luyện

Các chỉ số đánh giá mô hình YOLO sau khi huấn luyện

Sau khi huấn luyện mô hình YOLO, việc đánh giá hiệu suất là cần thiết để đảm bảo mô hình hoạt động tốt với dữ liệu thực tế. Các chỉ số đánh giá chính bao gồm:

• Precision (Độ chính xác)

Precision là tỷ lệ dự đoán dương đúng trên tổng số dự đoán dương (dương đúng + dương sai). Chỉ số này cho biết mô hình chính xác đến đâu khi dự đoán đối tượng tồn tại.

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

Hình 12: Precision

• Recall (Độ nhạy)

Recall là tỷ lệ dự đoán dương đúng trên tổng số đối tượng thực tế (dương đúng + âm sai). Chỉ số này cho biết mô hình có thể tìm được tất cả đối tượng thực tế trong tập dữ liệu tốt đến mức nào.

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

Hình 13: Recall

• **Intersection over Union (IoU)**

IoU là phép đo độ trùng lặp giữa hộp giới hạn dự đoán và hộp giới hạn thực tế, được tính bằng cách chia diện tích vùng giao nhau cho tổng diện tích của cả hai hộp. Một dự đoán được coi là chính xác nếu IoU của nó với hộp thực tế vượt quá ngưỡng nhất định, thường là 0.5.

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Total Area}}$$

Hình 14: Intersection over Union

• **mAP@50 (Mean Average Precision tại IoU 0.5)**

mAP@50 là trung bình chính xác trung bình được tính ở một ngưỡng IoU duy nhất là 0.5. Nghĩa là, một phát hiện được coi là đúng nếu IoU giữa hộp giới hạn dự đoán và hộp thực tế lớn hơn hoặc bằng 0.5.

$$\text{mAP@50} = \frac{1}{N} \sum_{i=1}^N \text{AP}_i \text{ at IoU } 0.5$$

Hình 15: mAP@50

• **mAP@50-95 (Mean Average Precision tại IoU từ 0.5 đến 0.95)**

mAP@50-95 cung cấp đánh giá chi tiết hơn về hiệu suất của mô hình trên các ngưỡng IoU khác nhau. Nó được tính bằng cách trung bình các giá trị AP ở nhiều ngưỡng IoU, từ 0.5 đến 0.95 với bước 0.05. Chỉ số này quan trọng trong các ứng dụng cần xác định chính xác vị trí đối tượng.

$$\text{mAP@50-95} = \frac{1}{10} \sum_{t=0.5}^{0.95} \left(\frac{1}{N} \sum_{i=1}^N \text{AP}_i \text{ at IoU } t \right)$$

Hình 16: mAP@50-95

• Kết quả Đánh giá Mô hình YOLO

```
120 epochs completed in 0.076 hours.
Optimizer stripped from runs/detect/train/weights/last.pt, 6.2MB
Optimizer stripped from runs/detect/train/weights/best.pt, 6.2MB

Validating runs/detect/train/weights/best.pt...
Ultralytics YOLOv8.2.48 Python-3.10.12 torch-2.3.0+cu121 CUDA:0 (Tesla T4, 15102MiB)
Model summary (fused): 168 layers, 3006428 parameters, 0 gradients, 8.1 GFLOPs

```

Class	Images	Instances	Box(P)	R	mAP50	mAP50-95
all	48	567	0.999	1	0.995	0.895
xe may	40	290	0.999	1	0.995	0.842
oto	47	241	0.999	1	0.995	0.912
xe bus	31	36	0.998	1	0.995	0.932

```
Speed: 0.2ms preprocess, 2.6ms inference, 0.0ms loss, 4.9ms postprocess per image
Results saved to runs/detect/train
```

PhotoSmile.vn

Hình 17: Kết quả huấn luyện

Hiệu suất tổng thể

- **Tất cả các lớp:**
 - Số lượng hình ảnh: 48
 - Số lượng đối tượng: 567
 - Độ chính xác (P): 0.999
 - Tỷ lệ hồi tưởng (R): 1
 - mAP@50: 0.995
 - mAP@50-95: 0.895

Hiệu suất theo lớp

- **Xe máy ("xe may"):**
 - Số lượng hình ảnh: 40
 - Số lượng đối tượng: 290
 - Độ chính xác (P): 0.999
 - Tỷ lệ hồi tưởng (R): 1
 - mAP@50: 0.995
 - mAP@50-95: 0.842
- **Ô tô ("oto"):**
 - Số lượng hình ảnh: 47
 - Số lượng đối tượng: 241
 - Độ chính xác (P): 0.999
 - Tỷ lệ hồi tưởng (R): 1
 - mAP@50: 0.995
 - mAP@50-95: 0.912
- **Xe buýt ("xe bus"):**
 - Số lượng hình ảnh: 31
 - Số lượng đối tượng: 36
 - Độ chính xác (P): 0.998
 - Tỷ lệ hồi tưởng (R): 1
 - mAP@50: 0.995
 - mAP@50-95: 0.93

Tổng kết

Mô hình đạt **độ chính xác và độ nhạy rất cao** đối với tất cả các lớp đối tượng, đặc biệt là xe máy và ô tô.

mAP@50-95 trên toàn bộ tập dữ liệu là 0.895, cho thấy mô hình phát hiện chính xác các đối tượng trên nhiều ngưỡng IoU khác nhau.

Thời gian xử lý rất nhanh với **2.0ms cho suy luận và 4.9ms cho hậu xử lý** mỗi hình ảnh, điều này làm cho mô hình YOLO rất hiệu quả cho các ứng dụng thời gian thực.

Thử nghiệm xác định đối tượng đã huấn luyện

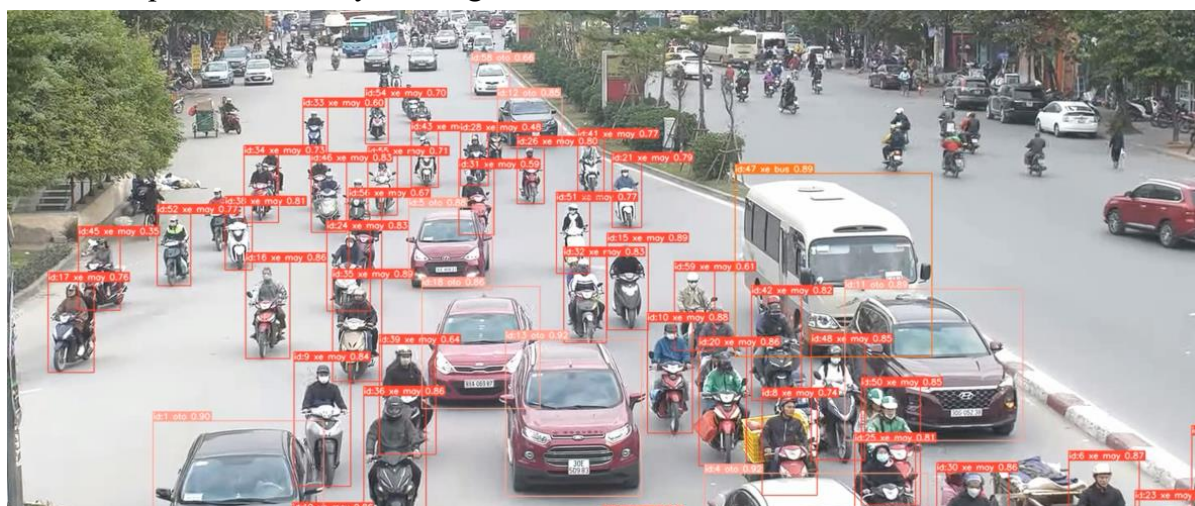
Để thực hiện phát hiện đối tượng với YOLOv8, ta cần sử dụng các thư viện hỗ trợ như Ultralytics. Dưới đây là ví dụ cơ bản để phát hiện đối tượng trong một hình ảnh sử dụng YOLOv8.

```
1 from ultralytics import YOLO
2 model = YOLO("custom2.pt") # pretrained YOLOv8n model
3 result = model("img1.png") # return Results objects
4 result.show() # display to screen
```

PhotoSmile.vn

Hình 18: Mã nguồn

Kết quả sau khi chạy chương trình:



Hình 19: Hình ảnh thử nghiệm

CHƯƠNG V: XÂY DỰNG WEB GIÁM SÁT

1. Giới thiệu chung

Trong chương này, nhóm em trình bày quá trình tích hợp mô hình nhận diện đối tượng (YOLO) vào một hệ thống Web nhằm phục vụ cho việc nhận diện xe cộ trong thời gian thực, đếm số lượng xe các loại. Việc tích hợp này cho phép người dùng dễ dàng truy cập, theo dõi và tương tác với hệ thống qua giao diện trình duyệt.

Hệ thống web được xây dựng nhằm trực quan hóa quá trình giám sát giao thông, bao gồm cả hình ảnh video đã được xử lý và dữ liệu định lượng như số lượng phương tiện theo từng loại xe (xe máy, ô tô, xe buýt, xe tải,...). Điều này giúp nâng cao tính ứng dụng của hệ thống, không chỉ trong nghiên cứu mà còn trong triển khai thực tế.

Kiến trúc hệ thống web bao gồm:

Frontend: giao diện hiển thị video và dữ liệu đếm xe theo thời gian thực.

Backend: xây dựng bằng Node.js, đảm nhận việc nhận dữ liệu từ mô hình YOLO (chạy bằng Python + OpenCV), lưu trữ vào cơ sở dữ liệu MySQL và cung cấp API cho frontend.

Toàn bộ quá trình hoạt động của hệ thống bao gồm: mô hình AI xử lý video đầu vào → đếm xe theo loại → gửi kết quả về backend → lưu dữ liệu → frontend lấy dữ liệu và hiển thị cho người dùng.

2. Thiết kế hệ thống Web giám sát

Hệ thống Web giám sát được thiết kế theo mô hình client-server, trong đó phần **backend** xử lý logic nghiệp vụ và cung cấp API, còn phần **frontend** hiển thị thông tin cho người dùng qua trình duyệt.

a. Kiến trúc hệ thống

Hệ thống bao gồm các thành phần chính:

- Camera giám sát: Ghi nhận hình ảnh giao thông tại một vị trí cố định theo thời gian thực.
- Mô hình AI (YOLOv8): Nhận diện phương tiện (xe máy, ô tô, xe buýt, xe tải) từ khung hình video.
- Flask server (Python): Chạy mô hình YOLO, xử lý khung hình, đếm số lượng phương tiện và gửi dữ liệu nhận diện về backend.
- Backend (Node.js + Express): Nhận dữ liệu từ Flask, lưu vào cơ sở dữ liệu MySQL và cung cấp API cho frontend truy xuất.
- Database (MySQL): Lưu trữ dữ liệu phương tiện nhận diện kèm timestamp.
- Frontend (HTML/CSS/JS hoặc React): Hiển thị video giám sát và bảng thống kê số lượng phương tiện theo thời gian thực.

b. Luồng dữ liệu

1. Camera truyền khung hình đến mô hình YOLO đang chạy trên Flask server.
2. Mỗi khung hình được xử lý để phát hiện và phân loại các phương tiện.
3. Số lượng phương tiện theo loại được gửi đến backend thông qua HTTP request.
4. Backend ghi dữ liệu vào bảng `vehicle` trong cơ sở dữ liệu MySQL.

5. Frontend định kỳ gọi API từ backend để lấy dữ liệu mới và cập nhật lên giao diện người dùng.

c. Cơ sở dữ liệu

Cấu trúc bảng vehicle:

id	motobike	car	bus	truck	timestamp
1	10	2	0	1	2025-05-14 10:30:00

3. Gửi luồng dữ liệu camera tới backend

Phần này được thực hiện bằng ngôn ngữ Python, có vai trò kết nối giữa mô-đun xử lý ảnh (nhận diện và đếm phương tiện giao thông từ video) và phần backend (Node.js). Sau khi đếm số lượng phương tiện từng loại, dữ liệu sẽ được gửi định kỳ đến máy chủ backend để lưu trữ.

a. Chức năng chính

Tạo dữ liệu dạng JSON từ kết quả đếm xe:

Số lượng các loại xe (xe máy, ô tô, xe buýt, xe tải) được lấy từ biến `vehicle_counts` – đây là biến đã được cập nhật liên tục trong quá trình xử lý video.

Gửi dữ liệu về server:

Sau mỗi khoảng thời gian `send_interval`, chương trình gửi dữ liệu đếm xe tới server backend thông qua HTTP POST với endpoint `http://localhost:3000/vehicles`.

Quản lý thời gian gửi dữ liệu:

Dữ liệu chỉ được gửi nếu thời gian hiện tại đã vượt qua thời điểm gửi gần nhất một khoảng `send_interval`. Việc này tránh gửi dữ liệu quá thường xuyên gây quá tải server.

Chuyển đổi và trả về khung hình dạng JPEG:

Khung hình video (frame) được mã hóa thành định dạng JPEG rồi chuyển thành chuỗi bytes để stream đến frontend – phục vụ cho phần hiển thị video theo thời gian thực.

b. Đoạn mã chính

```
if current_time - last_sent_time >= send_interval:
    data = {
        'motobike': vehicle_counts.get('xe máy', 0),
        'car': vehicle_counts.get('ô tô', 0),
        'bus': vehicle_counts.get('xe buýt', 0),
        'truck': vehicle_counts.get('xe tải', 0)
    }
    try:
        response = requests.post('http://localhost:3000/vehicles', json=data)
        print("Dữ liệu đã gửi:", response.json())
    except requests.exceptions.RequestException as e:
        print("Lỗi khi gửi dữ liệu:", e)
    last_sent_time = current_time
```

c. Lợi ích và vai trò

Đồng bộ hóa dữ liệu giữa frontend, backend và xử lý ảnh.

Đảm bảo hệ thống hoạt động ổn định theo thời gian thực, không gửi dữ liệu quá dày đặc.

Giúp lưu lại dữ liệu lịch sử số lượng phương tiện để phục vụ phân tích sau.

4. Xây dựng backend

Phần backend của hệ thống được xây dựng bằng Node.js kết hợp với MySQL để xử lý các yêu cầu từ phía người dùng và tương tác với cơ sở dữ liệu. Backend đóng vai trò trung gian giữa giao diện người dùng (frontend) và cơ sở dữ liệu, chịu trách nhiệm nhận yêu cầu từ phía client, truy xuất dữ liệu từ database, xử lý và trả về kết quả.

a. Công nghệ sử dụng

Ngôn ngữ lập trình: JavaScript (chạy trên môi trường Node.js)

Cơ sở dữ liệu: MySQL

Framework: Express.js

Thư viện hỗ trợ:

mysql2: dùng để kết nối và thực hiện truy vấn với cơ sở dữ liệu MySQL.

cors: cho phép truy cập tài nguyên giữa các nguồn khác nhau.

path: hỗ trợ xử lý đường dẫn tệp trên server.

b. Chức năng chính

Kết nối đến cơ sở dữ liệu MySQL: Tạo kết nối với database cameratrafic để thực hiện các thao tác truy xuất và lưu trữ dữ liệu.

Phục vụ file tĩnh: Sử dụng middleware express.static để phục vụ các tệp HTML, CSS, JS từ thư mục gốc, hỗ trợ cho phần frontend.

Giao tiếp với client qua API RESTful:

GET /vehicles: Lấy tất cả dữ liệu phương tiện đã được lưu, sắp xếp theo thời gian mới nhất.

POST /vehicles: Nhận dữ liệu mới (số lượng xe máy, ô tô, xe buýt, xe tải) từ frontend và lưu vào cơ sở dữ liệu kèm theo timestamp.

c. Quy trình hoạt động

Server Node.js được khởi chạy tại cổng 3000.

Khi người dùng truy cập địa chỉ `http://localhost:3000`, server sẽ gửi file `index.html` cho client.

Khi hệ thống frontend gửi dữ liệu đếm xe, server tiếp nhận yêu cầu qua route `/vehicles` và lưu dữ liệu vào MySQL.

Khi người dùng cần xem lại dữ liệu, frontend gửi yêu cầu GET, server sẽ truy xuất dữ liệu từ bảng vehicle và trả về định dạng JSON.

d. Kết luận

Phần backend đảm bảo hoạt động ổn định, dễ mở rộng và duy trì. Việc sử dụng Express.js giúp xây dựng API nhanh chóng và rõ ràng, trong khi MySQL cung cấp hệ quản trị cơ sở dữ liệu mạnh mẽ và phổ biến.

5. Giao diện web

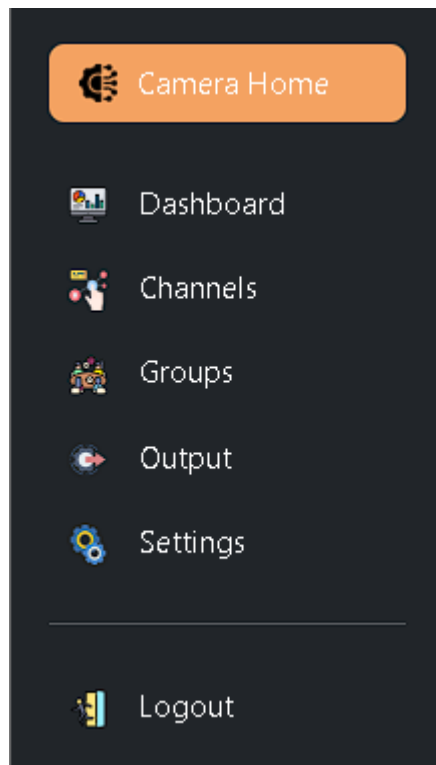
a. Trang dashboard

Giao diện dashboard là thành phần trung tâm giúp người dùng trực quan hóa dữ liệu, giám sát trạng thái camera và truy cập nhanh đến các chức năng chính của hệ thống. Giao diện được thiết kế hiện đại, đơn giản, dễ sử dụng, tương thích với nhiều thiết bị nhờ sử dụng Bootstrap và hỗ trợ biểu đồ bằng Chart.js.

Cấu trúc tổng thể của giao diện bao gồm các phần chính sau:

- **Sidebar (Thanh bên trái):** Thanh điều hướng cho phép người dùng truy cập nhanh đến các trang như:

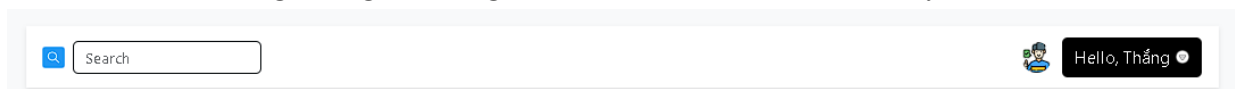
- + Dashboard: Trang chính hiển thị luồng dữ liệu.
- + Channels: Danh sách các camera riêng lẻ.
- + Groups: Hiển thị nhóm các camera theo địa điểm (ví dụ: một ngã tư).
- + Output: Hiển thị các ảnh đã lưu từ backend.
- + Settings và Logout.



Hình 20: Sidebar

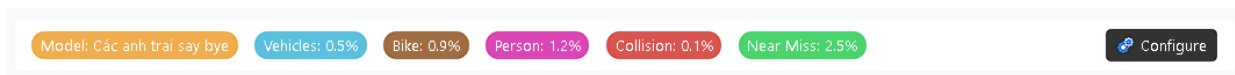
- **Topbar (Thanh trên cùng):** Bao gồm:

- + Khung tìm kiếm: Cho phép người dùng tìm kiếm thông tin.
- + Thông tin người dùng: Hiển thị avatar và lời chào tùy biến.



Hình 21: Topbar

- Model bar:



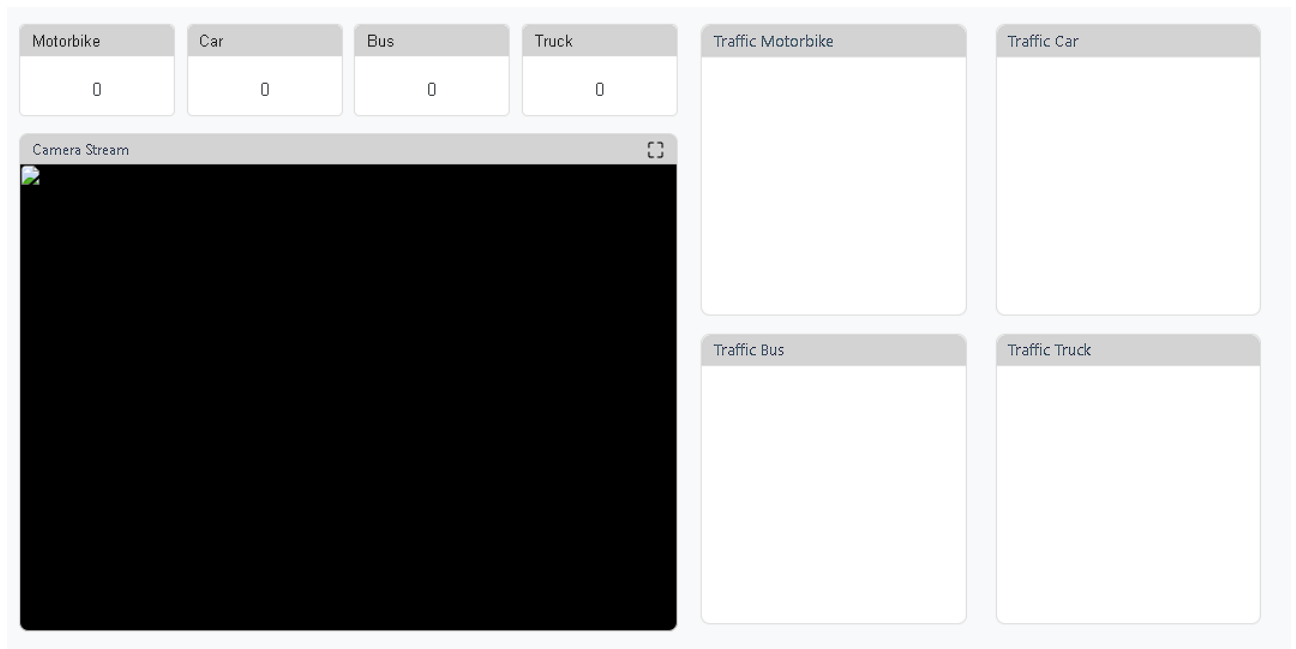
Hình 22: Model bar

Hiển thị thông tin thống kê tổng quát về mô hình AI đang chạy, bao gồm tỷ lệ phát hiện:

- + Vehicles
- + Bike
- + Person
- + Collision
- + Near miss

Ngoài ra, có nút "*Configure*" cho phép chuyển tới giao diện cấu hình hệ thống.

- Main content (Nội dung chính):



Hình 23: Dashboard

Gồm hai phần trái – phải:

+ **Left panel:**

- **Thẻ thống kê phương tiện:** Hiển thị số lượng xe máy (motorbike), ô tô (car), xe buýt (bus), và xe tải (truck). Các giá trị được cập nhật động qua DOM.
- **Khung stream:** Trình phát video từ camera được gắn vào thẻ <video> với ID cameraFeed.

+ **Right panel:**

- **Biểu đồ thống kê (sử dụng Chart.js):** Gồm các biểu đồ theo từng loại phương tiện:

- + Traffic Motorbike
- + Traffic Car

- + Traffic Bus
- + Traffic Truck

- Footer (info-bar):



Hình 24: Footer

Thanh thông tin cuối trang cung cấp:

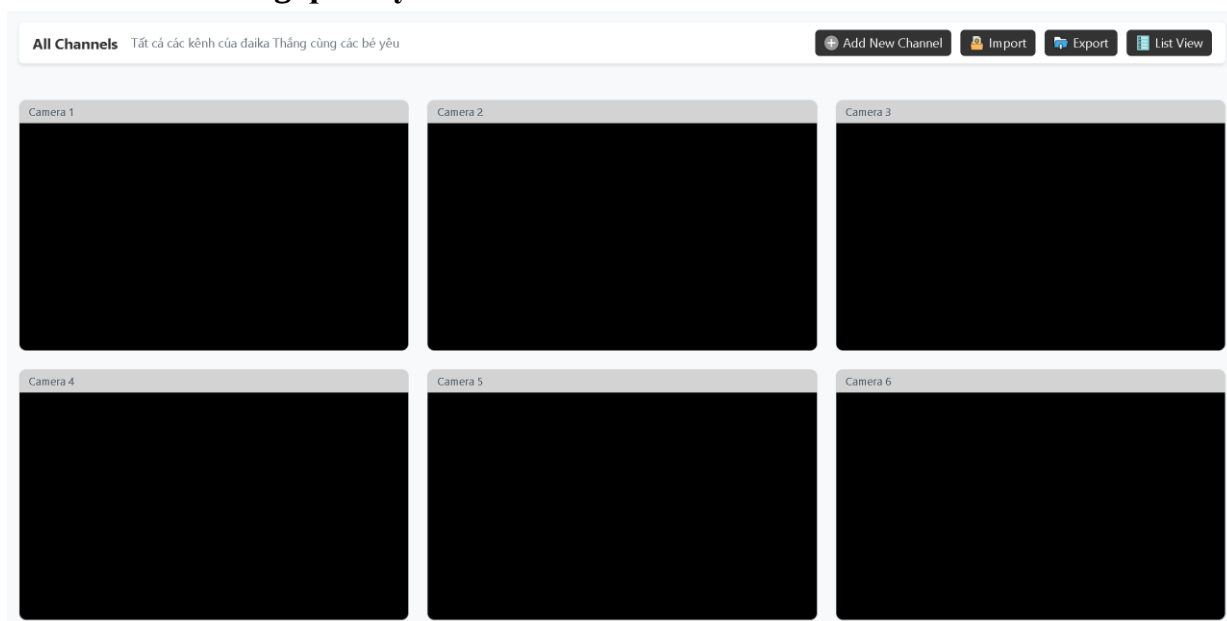
- + Tên nhóm thực hiện dự án
- + Các liên kết như *Privacy Policy*, *Terms and Conditions*, *Contact*, *Apps*

Trang giao diện sử dụng các thư viện ngoài thông qua CDN bao gồm:

- Bootstrap 5.3.0: Dùng để xây dựng bố cục và responsive UI.
- Font Awesome: Dùng cho icon sidebar và các biểu tượng chức năng.
- Chart.js: Vẽ biểu đồ tương tác và động.
- Custom CSS và JS: Tập styles.css và scripts.js để định nghĩa thêm các

hiệu ứng, logic cập nhật dữ liệu từ backend.

b. Trang quản lý camera.



Trang "Channels" trong hệ thống Camera Streams đóng vai trò quan trọng trong việc quản lý và hiển thị các kênh camera mà người dùng có thể theo dõi. Người dùng có thể thêm, sửa, xóa các kênh hoặc quản lý các thông tin liên quan đến các kênh camera đang được phát trực tuyến. Trang này được thiết kế để giúp người dùng dễ dàng tương tác và theo dõi video từ các camera.

Thanh quản lý kênh (Channel Management Bar):

- Add New Channel: Cho phép người dùng thêm một kênh camera mới vào hệ thống.
- Import: Cho phép người dùng nhập các kênh từ các hệ thống bên ngoài.
- Export: Cho phép người dùng xuất các thông tin về kênh ra ngoài hệ thống.
- List View: Cho phép người dùng chuyển đổi giữa chế độ xem danh sách hoặc chế độ xem dạng grid.

Danh sách kênh camera (Camera List):

- Mỗi kênh camera sẽ có một video stream, người dùng có thể xem trực tiếp các video từ các camera này.

- Các kênh camera được liệt kê với tiêu đề rõ ràng.

Tính năng chính:

- Thêm kênh mới (Add New Channel):

- + Người dùng có thể thêm các kênh camera mới vào hệ thống bằng cách cung cấp thông tin cần thiết như tên kênh, nguồn video, v.v.

- Nhập và xuất kênh (Import and Export):

- + Các kênh camera có thể được nhập từ các nguồn dữ liệu khác, giúp người dùng dễ dàng thêm kênh mà không phải tạo mới hoàn toàn.

- + Người dùng cũng có thể xuất các kênh ra ngoài hệ thống dưới dạng tệp tin.

- Xem kênh (View Channels):

- + Mỗi kênh camera sẽ được hiển thị với một video stream trực tiếp, người dùng có thể xem video từ nhiều camera cùng một lúc.

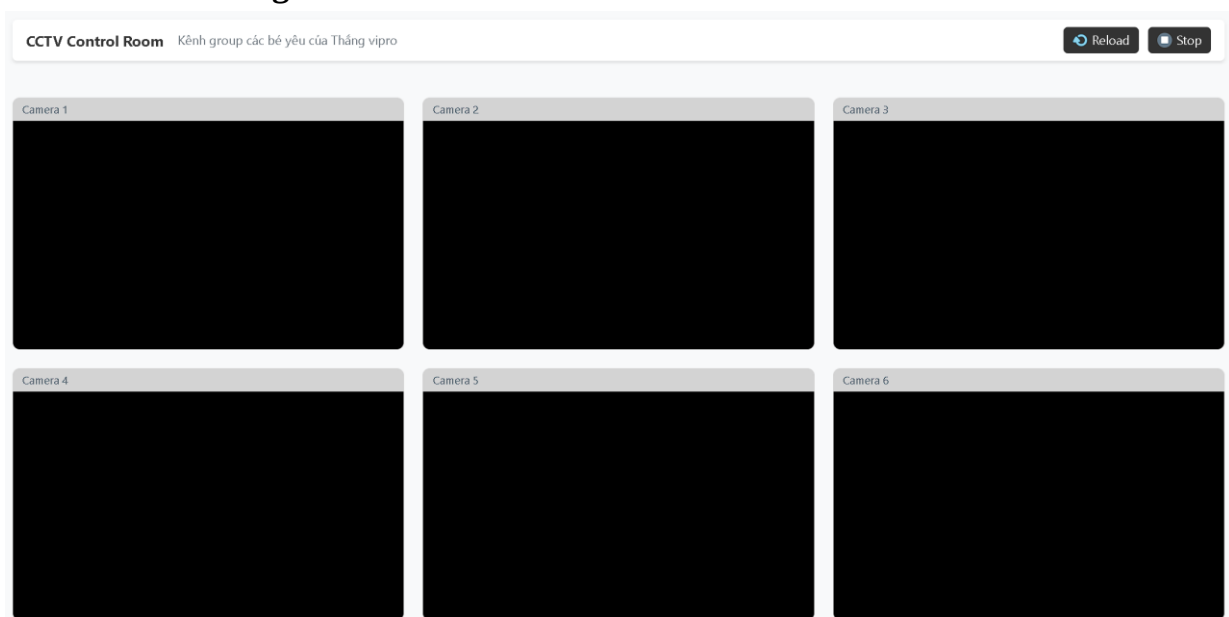
- Quản lý các kênh (Manage Channels):

- + Người dùng có thể quản lý thông tin của các kênh, bao gồm sửa đổi, xóa kênh hoặc thay đổi cấu hình.

Trang "Channels" được thiết kế để có thể xử lý nhiều kênh camera một cách hiệu quả. Giao diện người dùng hoạt động mượt mà, giúp người dùng dễ dàng thêm, quản lý, và xem các kênh camera. Các video stream được phát trực tiếp mà không gặp phải hiện tượng giật lag khi kết nối internet ổn định.

Trang "Channels" đóng vai trò quan trọng trong việc quản lý các kênh camera trong hệ thống Camera Streams. Nó cung cấp một cách tiếp cận dễ dàng và hiệu quả để người dùng có thể thêm mới, xem và quản lý các kênh camera, giúp nâng cao trải nghiệm người dùng trong việc giám sát và theo dõi video trực tuyến.

c. Trang xem camera theo nhóm



Trang groups.html là một phần quan trọng trong hệ thống giám sát camera, đặc biệt dành cho việc theo dõi và quản lý nhiều camera trong cùng một nhóm. Mục tiêu chính của trang này là cung cấp cho người dùng một giao diện trực quan, dễ dàng theo dõi và kiểm soát các luồng video từ các camera giám sát giao thông. Sau đây là phân tích chi tiết về cấu trúc và các tính năng của trang web này.

Nội dung chính:

- Thanh Điều Khiển (CCTV Control Room): Đây là phần quan trọng để điều khiển các luồng camera trong nhóm.

- + Reload: Nút này giúp làm mới các luồng camera, giúp đảm bảo rằng người dùng có thể nhận được hình ảnh mới nhất.

- + Stop: Nút này cho phép người dùng dừng tất cả các luồng camera đang phát trực tuyến, có thể hữu ích trong trường hợp cần ngừng theo dõi tạm thời.

- Camera Streams: Phần chính của trang là các luồng video trực tuyến từ các camera giám sát. Trang hiển thị 6 camera khác nhau, mỗi camera được trình bày trong một khối riêng biệt với các thông tin sau:

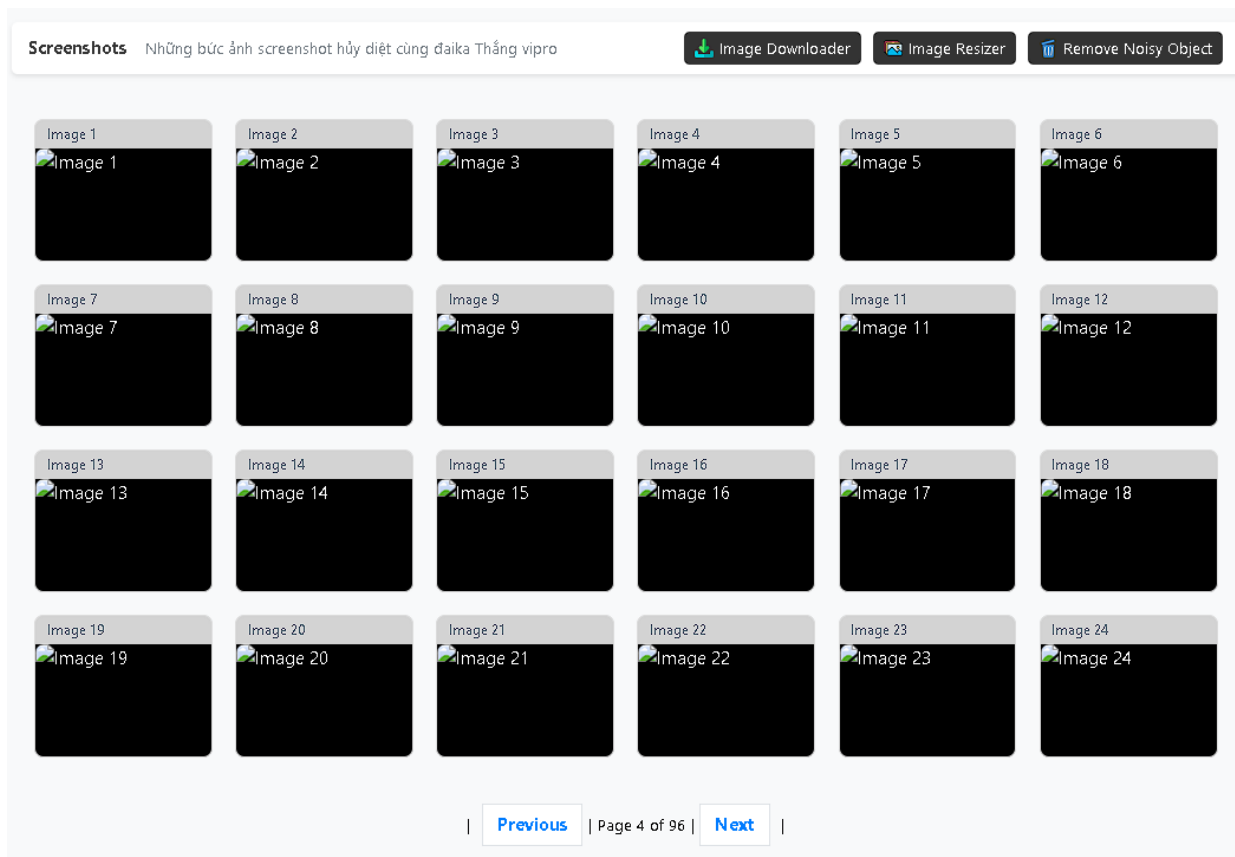
- + Tiêu Đề Camera: Mỗi camera có một tiêu đề đơn giản. Tiêu đề này giúp người dùng dễ dàng phân biệt giữa các camera.

- + Luồng Video: Mỗi camera được hiển thị qua thẻ <video>, hỗ trợ các thuộc tính như autoplay, playsinline, và muted, giúp luồng video được phát tự động và không có âm thanh. Hình ảnh được hiển thị trong chế độ đầy đủ chiều rộng và chiều cao để đảm bảo tỷ lệ khung hình phù hợp với các vùng hiển thị.

Các video này có thể là video trực tiếp từ các camera giám sát giao thông hoặc các camera đã được thiết lập sẵn để phát hình ảnh từ các giao lộ. Mỗi camera có thể được xem ở chế độ toàn màn hình nếu cần thiết, mang lại trải nghiệm tối ưu cho người dùng.

Trang groups.html cung cấp một giao diện trực quan và dễ sử dụng cho người dùng để theo dõi và quản lý các camera trong nhóm. Các tính năng như thanh điều khiển, luồng camera trực tiếp, phân trang và thông tin người dùng đều được thiết kế để nâng cao hiệu quả công việc trong các trung tâm giám sát giao thông. Các yếu tố giao diện và trải nghiệm người dùng được tối ưu hóa để đảm bảo rằng người dùng có thể dễ dàng điều khiển và theo dõi tình hình giao thông một cách nhanh chóng và hiệu quả.

d. Trang xem các hình ảnh camera chụp lại



Trang output.html là một thành phần quan trọng trong hệ thống giám sát camera, cung cấp các dữ liệu đầu ra sau khi các mô hình xử lý được áp dụng lên các luồng video từ các camera giám sát giao thông. Mục tiêu của trang này là hiển thị kết quả và thông tin quan trọng liên quan đến các sự kiện được phát hiện từ các camera, hỗ trợ người dùng trong việc theo dõi và phân tích dữ liệu hiệu quả. Sau đây là phân tích chi tiết về cấu trúc và các tính năng của trang output.html.

Nội dung chính:

- Các kết quả đầu ra: Phần chính của trang là nơi hiển thị các kết quả từ các mô hình xử lý được áp dụng lên các luồng video từ các camera. Các kết quả này là chụp màn hình mỗi 10s.
- Thông tin chi tiết: Các dữ liệu liên quan đến sự kiện, như thời gian, vị trí, và trạng thái của các camera.
- Dữ liệu được xử lý: Các hình ảnh hoặc video đã qua xử lý có thể được hiển thị hoặc tải xuống, phục vụ cho việc phân tích và báo cáo.

Các liên kết đầu ra:

Trang output.html có thể bao gồm các liên kết để tải về hoặc xem các dữ liệu xử lý. Các loại đầu ra có thể phát triển lên thành các hình ảnh từ camera sau khi đã qua các thuật toán xử lý, chẳng hạn như nhận diện khuôn mặt, nhận diện biển số xe, v.v.

Phân trang:

Trang output.html cũng bao gồm chức năng phân trang giúp người dùng dễ dàng di chuyển qua các kết quả xử lý. Các phân trang này hiển thị số lượng các kết quả xử

lý. Các liên kết Previous và Next cho phép người dùng quay lại hoặc tiếp tục đến các trang sau.

Giao diện của trang output.html được thiết kế để dễ sử dụng và tối ưu hóa cho trải nghiệm người dùng. Một số yếu tố trong thiết kế giao diện bao gồm:

- Bootstrap: Được sử dụng để đảm bảo tính linh hoạt và đáp ứng với các thiết bị di động.
- Font awesome: Cung cấp các biểu tượng giúp tăng tính dễ hiểu và dễ sử dụng cho các chức năng trong trang.
- Bố cục rõ ràng: Các thông tin và kết quả được trình bày theo dạng bảng hoặc danh sách, giúp người dùng dễ dàng theo dõi và kiểm tra các dữ liệu đầu ra.
- Tính năng tìm kiếm và bộ lọc: Giúp người dùng dễ dàng tìm kiếm các kết quả hoặc lọc theo các tiêu chí cụ thể, chẳng hạn như thời gian, loại sự kiện hoặc camera.

Trang output.html đóng vai trò quan trọng trong việc cung cấp các kết quả xử lý và phân tích từ các luồng video giám sát giao thông. Các tính năng như tìm kiếm, phân trang và hiển thị kết quả xử lý giúp người dùng có thể theo dõi và xử lý các sự kiện từ camera một cách hiệu quả. Giao diện được thiết kế rõ ràng, dễ sử dụng và tối ưu cho mọi thiết bị, mang đến trải nghiệm người dùng tốt nhất khi làm việc với các dữ liệu đầu ra của hệ thống giám sát.

CHƯƠNG VI: KẾT QUẢ

1. Kết quả đạt được

a. Về mặt lý thuyết

Nắm vững nguyên lý hoạt động của thuật toán YOLO (You Only Look Once), đặc biệt là phiên bản **YOLOv8** – một trong những mô hình mạnh mẽ và mới nhất dùng trong nhận diện đối tượng thời gian thực.

Tìm hiểu cấu trúc và cơ chế hoạt động của YOLOv8 như: Backbone, Neck, Head và cách hoạt động của các anchor, bounding box, confidence score.

Đã nghiên cứu cách tối ưu hóa YOLOv8 để phù hợp với môi trường giao thông Việt Nam như:

- Nhận diện được các loại xe phổ biến: xe máy, ô tô, xe tải, xe buýt.
- Làm việc tốt trong điều kiện ánh sáng thay đổi, thời tiết xấu, hình ảnh có nhiều đối tượng chồng lấp.

Tìm hiểu về quy trình xử lý ảnh/video từ camera giám sát, bao gồm các bước:

- Tiền xử lý ảnh
- Trích xuất khung hình từ video
- Kết nối với webcam hoặc camera IP

b. Về mặt thực tiễn

Xây dựng thành công mô hình nhận diện xe cộ sử dụng YOLOv8:

- Có thể phát hiện, phân loại xe máy, ô tô, xe tải,... từ ảnh hoặc video đầu vào.
- Độ chính xác trung bình đạt khoảng 95–99% trong điều kiện hình ảnh rõ nét.

Tích hợp thành công mô hình vào hệ thống web:

- Người dùng có thể tải ảnh hoặc video lên qua trình duyệt web.
- Backend xử lý bằng Flask, gọi mô hình YOLOv8 để dự đoán, lưu kết quả và trả về ảnh đã nhận diện.

- Giao diện đơn giản, thân thiện, dễ sử dụng.

Hệ thống hoạt động ổn định, cho phép xử lý ảnh trong thời gian ngắn (tùy thuộc vào kích thước ảnh và cấu hình máy chủ).

Có thể mở rộng xử lý thời gian thực nếu kết nối với webcam hoặc camera giám sát.

2. Hạn chế

- Độ chính xác giảm trong điều kiện ánh sáng yếu, mưa lớn hoặc ảnh mờ (do camera kém chất lượng).

- Khi các phương tiện che khuất nhau, hệ thống có thể bỏ sót hoặc gộp nhầm đối tượng.

- Thời gian xử lý ảnh/video có thể tăng nếu chạy trên máy cấu hình yếu hoặc sử dụng nhiều người truy cập cùng lúc.

- Giao diện web hiện tại mới ở mức cơ bản, chưa có chức năng thống kê hoặc lưu trữ lịch sử.

3. Hướng phát triển

Tối ưu mô hình YOLOv8 để cải thiện độ chính xác trong điều kiện ánh sáng yếu và đối tượng bị che khuất.

Tích hợp thêm tính năng nhận diện biển số xe nhằm phục vụ cho các bài toán như kiểm soát xe vào/ra, thu phí tự động.

Nâng cấp giao diện web:

- Thêm chức năng xem lịch sử, xuất báo cáo
- Thêm biểu đồ thống kê theo loại xe hoặc thời gian

Phát triển hệ thống nhận diện thời gian thực từ camera IP/webcam, hỗ trợ giám sát trực tiếp.

Triển khai mô hình trên cloud hoặc GPU server để xử lý nhanh hơn, phục vụ nhiều người dùng đồng thời.

Kết hợp với các công nghệ IoT, ví dụ: cảnh báo phương tiện lạ qua mạng, tích hợp vào hệ thống đèn giao thông thông minh,...

DANH MỤC TÀI LIỆU THAM KHẢO

- [1] X. Cong, S. Li, C. Fankai, C. Liu and M. Yue, A Review of YOLO Object Detection Algorithms based on Deep Learning, China : Tianjin University of Technology and Education, Tianjin 300222, 2023.
- [2] p. d. khanh, "YOLO You Only Look Once," 09 03 2020. [Online]. Available: <https://phamdinhhkhanh.github.io/2020/03/09/DarknetAlgorithm.html>.
- [3] R. Joseph and F. Ali, YOLO9000: Better, Faster, Stronger, Washington: University of Washington, 2016.
- [4] R. Joseph, D. Santosh, G. Ross and F. Ali, You Only Look Once: Unified, Real-Time Object Detection, Washington: University of Washington , Allen Institute for AI , Facebook AI Research, 2016.
- [5] H. Jizhou , Z. Zhenhai , G. Xueshan, L. Kejie and K. Xiao, Research on Negative Obstacle Detection Method Based on Image Enhancement and Improved Anchor Box YOLO, Guilin, Guangxi, China, 2022.
- [6] M. Đ. T. Thịnh, PHƯƠNG PHÁP CẢI TIẾN HỆ THỐNG NHẬN DẠNG XE CỘ BẰNG ỨNG DỤNG YOLO, Việt Nam: Tạp chí khoa học và công nghệ Trường Đại học Công nghiệp Thành Phố Hồ Chí Minh, 2024.
- [7] G. James, "How to Train an Ultralytics YOLOv8 Oriented Bounding Box (OBB) Model," 2024. [Online]. Available: <https://blog.roboflow.com/train-yolov8-obb-model/>.